Session 1: Messages API, Streaming and SDK Setup

Session Plan

#	Block	Time	Format
1	Exam orientation	10 min	Talk + discussion
2	Messages API	25 min	Concept → demo → quiz
3	SDK setup (auth, config, multi-turn state)	25 min	Concept → demo → quiz
4	Streaming	25 min	Concept → demo → quiz
5	Hands-on lab	30 min	Pairs, scaffolded starter
6	Wrap-up	5 min	Recap + cheat sheet

One client, built across the session

Module	Capability added to the client
Messages API	One `messages.create` call that prints the reply and `stop_reason`
SDK setup	API key from environment, plus a history list for multi-turn chat
Streaming	The same call switched to streaming, printing tokens as they arrive
Lab	System prompt, `max_tokens` handling, exam-style questions

Before the session

- [ ] Python 3.9+ (or Node 18+) installed
- [ ] `pip install anthropic python-dotenv`
- [ ] API key created at the Anthropic Console (it is shown only once, so copy it immediately)
- [ ] `.env` file containing `ANTHROPIC_API_KEY="..."`, with `.env` listed in `.gitignore`
- [ ] A test script that makes one call, so key problems are fixed before class

1. Exam Orientation

Goals

- Know what the exam covers and how questions are structured.
- Learn the approach for multiple-response questions, which you practise after every module.

Blueprint and question formats

- Map this session's topics to the exam domains: API requests, streaming, SDK usage and conversation state.
- Questions are either **single answer** or **multiple response** ("select all that apply", "select two").

Multiple-response tactics

1. Read the stem for the exact number of answers required.
2. Judge each option **independently** as true or false for the scenario.
3. Eliminate options that contradict documented behaviour, such as sending the key from client code, or expecting Claude to remember earlier turns.
4. Watch for absolutes ("always", "never"). They are frequently wrong, but not every time.
5. Re-read the stem after choosing, to check your answers fit the scenario and not just a general fact.

Warm-up

An option says "Claude remembers earlier requests in the same session". Is it true?

```
[(X)] No, the API is stateless, so you must resend the history
[( )] Yes, Claude keeps a server-side session
[( )] Only if streaming is enabled
***

Every request is independent. You keep the message list in your own code and send
all of it every turn.

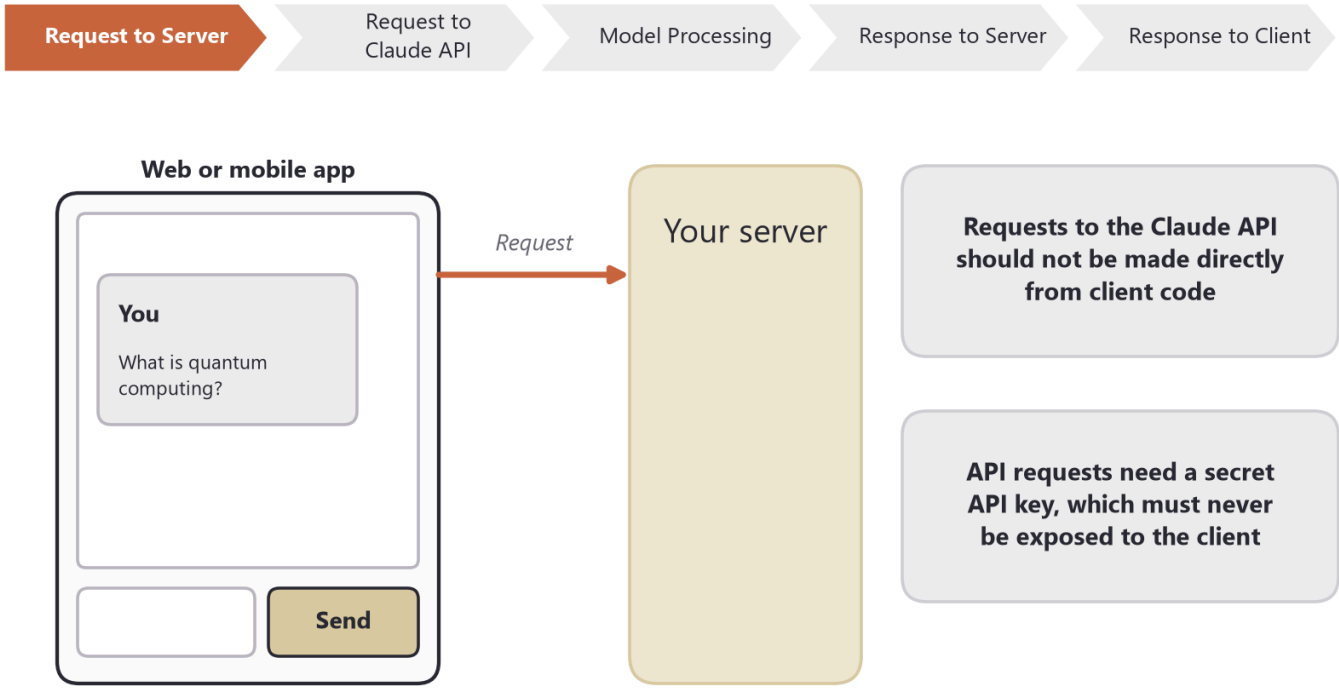
***
```

2. Messages API

The request lifecycle

Every interaction follows five steps:

1. The client sends the request to **your server**.
2. Your server sends the request to the **Anthropic API**.
3. The model processes it.
4. The API responds to your server.
5. Your server responds to the client.



Why you need a server: the API key is secret. Putting it in client-side code lets anyone extract it and make requests on your account. The key stays on your server.

Required request fields

Field	Purpose
API key	Identifies and authenticates the request (sent via the SDK or the `x-api-key` header)
`model`	Which Claude model to use
`messages`	The conversation, as a list of messages
`max_tokens`	A **safety ceiling** on generated tokens, not a target. Claude stops early if it can.

```
from dotenv import load_dotenv
load_dotenv()

from anthropic import Anthropic

client = Anthropic()          # reads ANTHROPIC_API_KEY from the environment
model = "claude-sonnet-4-0"   # use a current model ID from the models overview

def text_of(message):
    """Join the text blocks. content[0] is not always text: it can be a thinking
or tool_use block."""
    return "".join(b.text for b in message.content if b.type == "text")

message = client.messages.create(
    model=model,
    max_tokens=1000,
    messages=[{"role": "user", "content": "What is quantum computing? Answer in
```

```
one sentence"}],
)
print(text_of(message))
```

> **Why a helper and not `message.content[0].text`?** `content` is a list of blocks. Current models think before they answer by default, so `content[0]` can be a `thinking` block, which has no `.text` (you get `'ThinkingBlock' object has no attribute 'text'`). `text_of` joins only the text blocks, so it works either way. Thinking tokens also count toward `max_tokens`, so a very small limit can leave no room for any text.

Roles and the system prompt

- Only two roles appear in `messages`: **`user`** (written by humans) and **`assistant`** (written by Claude).
- Messages **alternate** between `user` and `assistant`.
- The **system prompt is not a role**. It is a separate top-level `system` parameter.

```
system_prompt = """
You are a patient math tutor.
Do not directly answer a student's questions.
Guide them to a solution step by step.
"""

client.messages.create(model=model, max_tokens=1000, messages=messages,
system=system_prompt)
```

> **Gotcha:** the API does not accept `system=None`. Only include the parameter when you have a value.

```
def chat(messages, system=None):
    params = {"model": model, "max_tokens": 1000, "messages": messages}
    if system:
        params["system"] = system
    message = client.messages.create(**params)
    return text_of(message)
```

Worked example: system prompt and `max_tokens`

One question, three calls. The outputs are **illustrative** sample values, so your wording will differ.

```
def show(message):
    print(text_of(message))
    print("stop_reason:", message.stop_reason, "| output_tokens:",
message.usage.output_tokens)

question = [{"role": "user", "content": "How do I solve 5x + 2 = 3 for x?"}]
```

A. No system prompt

```
show(client.messages.create(model=model, max_tokens=1000, messages=question))
```

```
Subtract 2 from both sides: 5x = 1.
Then divide both sides by 5: x = 1/5, or 0.2.
stop_reason: end_turn | output_tokens: 38
```

B. With a tutor system prompt

```
show(client.messages.create(
    model=model, max_tokens=1000, messages=question,
    system="You are a patient math tutor. Do not directly answer. Guide step by
step.",
))
```

```
Good question! What could you do to both sides of the equation to get the
term with x on its own? Try that first and tell me what you get.
stop_reason: end_turn | output_tokens: 34
```

C. Same as A, but `max_tokens=15`

```
show(client.messages.create(model=model, max_tokens=15, messages=question))
```

```
To solve 5x + 2 = 3 for x, start by subtracting 2 from
stop_reason: max_tokens | output_tokens: 15
```

What to notice:

- The system prompt changes **behaviour**, not the question. B guides and A answers.
- In C the reply stops mid-sentence and `stop_reason` is `max_tokens`. There is no error, so you must check the field yourself.
- `output_tokens` equals the limit exactly when the limit was hit.
- On a model that thinks first, the reply in C can even be **empty**: thinking tokens count toward `max_tokens`, so a tiny limit is spent before any text. `stop_reason` is still `max_tokens`.

The response object

Field	What it holds
`id`	Unique message ID (starts with `msg_`)
`type`	Always `"message"`
`role`	Always `"assistant"`
`content`	A **list of content blocks**, not a string. For plain text, join the `text` blocks (`text_of` above): `content[0]` can be a `thinking` block. See the block types below.
`model`	The model that responded
`stop_reason`	Why generation ended (see the next section)
`stop_sequence`	The custom stop string that was matched, or `null`
`usage`	Token counts: `input_tokens` and `output_tokens`, which you are billed on. With prompt caching it also has `cache_creation_input_tokens` and `cache_read_input_tokens`.

Content block types

Both requests and responses carry content as **blocks**. In a request, a plain string is shorthand for one `text` block. Never assume `content[0]` is text. Check `block.type` first.

Blocks Claude returns in `response.content`

Block `type`	Meaning	When you see it
`text`	Generated text (`.text`), optionally with `citations`	Almost every reply
`tool_use`	A call to **your** tool: `id`, `name` and `input`	`stop_reason == "tool_use"`
`thinking`	Claude's reasoning (`.thinking` plus a `.signature`)	Extended thinking enabled
`redacted_thinking`	Encrypted reasoning that is not readable	Extended thinking, when the reasoning is redacted
`server_tool_use`	A call to a tool that **Anthropic runs** (web search, web fetch, code execution)	Server tools enabled
`web_search_tool_result`	Results of a server-side web search	After `server_tool_use` for web search
`web_fetch_tool_result`	Content fetched from a URL by the server	After `server_tool_use` for web fetch

Block `type`	Meaning	When you see it
`code_execution_tool_result`	Output of server-side code execution	Code execution tool enabled
`bash_code_execution_tool_result`	Output of a server-side bash run	Code execution tool enabled
`text_editor_code_execution_tool_result`	Output of a server-side text editor run	Code execution tool enabled
`tool_search_tool_result`	Results of a tool search	Tool search enabled
`mcp_tool_use` / `mcp_tool_result`	A call to a remote MCP server tool, and its result	MCP connector enabled

Blocks you send in `messages[].content`

Block `type`	Meaning
`text`	Plain text
`image`	An image from `base64`, `url` or an uploaded `file` (JPEG, PNG, GIF, WebP)
`document`	A PDF or plain-text document, optionally with `title`, `context` and `citations`
`search_result`	A search result with source, title and text, used for citations
`tool_result`	The result of a **client** tool call. It references `tool_use_id` and can set `is_error`.
`tool_use`	A previous tool call, replayed from the assistant turn
`thinking` / `redacted_thinking`	Earlier reasoning, passed back **unmodified** in multi-turn conversations
`server_tool_use` and the `*_tool_result` blocks	Earlier server-tool activity, replayed from the assistant turn
`container_upload`	A file to place in the code-execution container (`file_id`)

> **Rule of thumb:** `tool_use` means *you* run the tool and reply with a `tool_result`. `server_tool_use` means *Anthropic* already ran it, and the result arrives in the same response.

```
for block in message.content:
    if block.type == "text":
        print(block.text)
    elif block.type == "tool_use":
        print("Claude wants to call", block.name, "with", block.input)
    elif block.type == "thinking":
        print("[reasoning]", block.thinking)
```

Worked example: request parameters and response

The three examples below are **illustrative**. The text, IDs and token counts are sample values, so your own output will differ. The structure is what matters.

Request parameters

Parameter	Required	What it does
`model`	yes	Which model answers
`max_tokens`	yes	Ceiling on generated tokens
`messages`	yes	The conversation (alternating `user` / `assistant`)
`system`	no	System prompt, top-level (never `None`)
`temperature`	no	Randomness, `0.0`–`1.0` (default `1.0`)
`top_p`	no	Nucleus sampling. Change this **or** `temperature`, not both. Some newer models reject both together.
`top_k`	no	Sample only from the top K tokens
`stop_sequences`	no	Custom strings that stop generation
`stream`	no	`true` returns server-sent events instead of one JSON body
`metadata`	no	`{"user_id": "..."}` as an opaque ID for abuse tracking. Do not put personal data in it.
`service_tier`	no	`"auto"` or `"standard_only"` for priority-capacity handling
`tools` / `tool_choice`	no	Tool definitions, and how Claude may pick them
`thinking`	no	Extended thinking: `{"type": "adaptive"}` on current models (older models: `{"type": "enabled", "budget_tokens": N}`)

Example 1: sampling, stop sequences and metadata

```
message = client.messages.create(
    model="claude-sonnet-4-0",
    max_tokens=1000,
    system="You are a concise API instructor.",
    messages=[
        {"role": "user", "content": "List three required request fields, then
write ###."},
    ],
    temperature=0.3,
    top_k=40,
    stop_sequences=["###"],
    metadata={"user_id": "learner-042"},
    service_tier="auto",
```

```
)

print(message.model_dump_json(indent=2))
```

Output:

```
{
  "id": "msg_01ABCdefGHIjklMNOpqrSTUv",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-20250514",
  "content": [
    {
      "type": "text",
      "text": "1. model\n2. max_tokens\n3. messages\n"
    }
  ],
  "stop_reason": "stop_sequence",
  "stop_sequence": "###",
  "usage": {
    "input_tokens": 31,
    "output_tokens": 19,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0,
    "service_tier": "standard"
  }
}
```

What to notice:

- `stop_reason` is `"stop_sequence"` and `stop_sequence` names the string that matched.
- The stop string `###` is **not** included in the text.
- `usage` is how you track cost. Input tokens include the system prompt.
- Current models reject non-default `temperature`, `top_p` and `top_k` with a 400. If you see that error, drop them.

Example 2: tools and `tool_choice`

```
message = client.messages.create(
    model="claude-sonnet-4-0",
    max_tokens=1000,
    tools=[
        {
            "name": "get_weather",
            "description": "Get the current weather for a city.",
            "input_schema": {
                "type": "object",
                "properties": {"city": {"type": "string"}},
```

```
                "required": ["city"],
            },
        }
    ],
    tool_choice={"type": "auto"},
    messages=[{"role": "user", "content": "What's the weather in Chennai?"}],
)
```

Output:

```
{
  "id": "msg_01XYZabcDEFghiJKLmnoPQr",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-20250514",
  "content": [
    { "type": "text", "text": "I'll check the weather in Chennai." },
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq9178351q9",
      "name": "get_weather",
      "input": { "city": "Chennai" }
    }
  ],
  "stop_reason": "tool_use",
  "stop_sequence": null,
  "usage": { "input_tokens": 312, "output_tokens": 64 }
}
```

What to notice:

- `content` holds **two blocks**, a `text` block and a `tool_use` block. This is why you loop over blocks and never assume `content[0]` is the whole answer.
- `stop_reason` is `"tool_use"`. Your code runs the tool and sends back a `tool_result` that references the same `id`.

Example 3: extended thinking

```
message = client.messages.create(
    model="claude-sonnet-4-0",
    max_tokens=2000,
    thinking={"type": "adaptive", "display": "summarized"},
    messages=[{"role": "user", "content": "Is 391 a prime number?"}],
)
```

Current models use **adaptive** thinking: the model decides how much to think, and `budget_tokens` is rejected with a 400. (Older models used `{"type": "enabled", "budget_tokens": N}`, with `N` at least 1024 and

below `max_tokens`.) `display: "summarized"` makes the thinking text readable. The default, `"omitted"`, leaves it empty, and it is billed either way. Thinking does not combine with a custom `temperature` or `top_k`.

Output:

```
{
  "id": "msg_01THINKexampleID0000000",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-20250514",
  "content": [
    {
      "type": "thinking",
      "thinking": "391 is not even. Try 17: 17 x 23 = 391.",
      "signature": "EuYBCkQYAiJA...(long opaque string)"
    },
    {
      "type": "text",
      "text": "No. 391 = 17 x 23, so it is not prime."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": { "input_tokens": 45, "output_tokens": 112 }
}
```

What to notice:

- The `thinking` block comes **before** the `text` block.
- If you continue the conversation, pass thinking blocks back **unmodified**.
- Thinking tokens count toward `output_tokens` **and** toward `max_tokens`.
- Current models think **by default**, even without a `thinking` parameter. That is why `content[0]` is not always text.

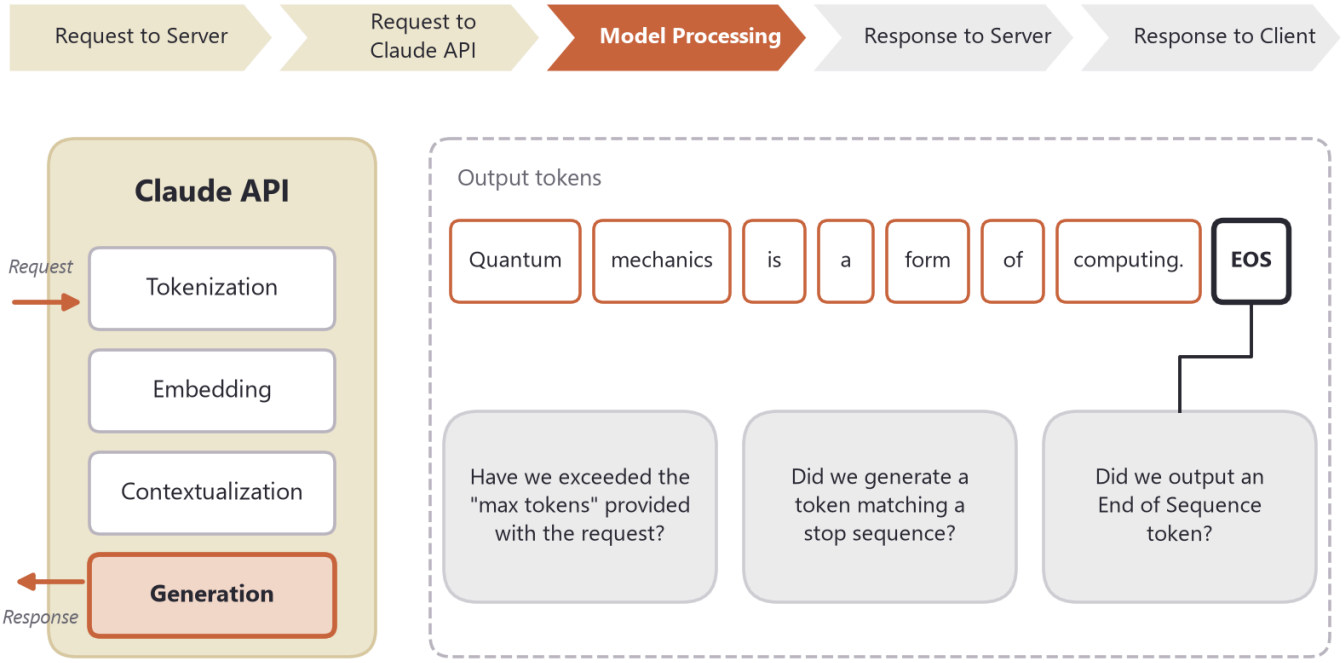
Reading the fields in code

```
print(message.id)             # msg_01ABC...
print(message.stop_reason)    # "stop_sequence"
print(message.stop_sequence)  # "###"
print(message.usage.input_tokens, message.usage.output_tokens)

text = "".join(b.text for b in message.content if b.type == "text")
```

`stop_reason`: why Claude stopped

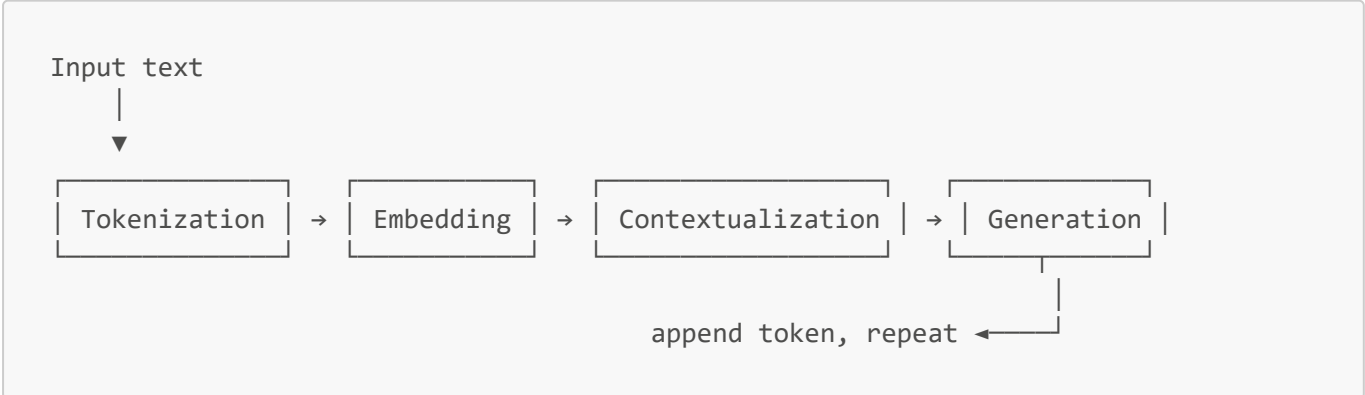
After each token Claude checks whether to continue.



`stop_reason`	**Meaning**	**What your code should do**
`end_turn`	Claude finished naturally	Use the response
`max_tokens`	Hit your limit, so output is **truncated**	Raise the limit, or continue the generation
`stop_sequence`	Hit a custom stop string you supplied	Use the response up to the stop point
`tool_use`	Claude wants to call a tool	Run the tool and send back the result
`pause_turn`	A server-tool turn was paused before it finished	Send the response back as-is to let Claude continue
`refusal`	Claude declined to respond for safety reasons	Show a fallback message, and do not retry unchanged

> **Live demo:** set `max_tokens=20` on a long question. Show the cut-off reply and `stop_reason == "max_tokens"`.

Inside Claude's processing (brief)



Claude picks each next token from a probability distribution (not always the top one), appends it, and repeats.

Quiz: Messages API

Q1. A reply ends mid-sentence and `stop_reason` is `max_tokens`. What is the correct interpretation?

```
[( )] Claude refused to answer
[(X)] The reply hit the `max_tokens` ceiling and was truncated
[( )] The API key lacks permission
[( )] The conversation history was too long
[[?]] Look at what `max_tokens` is for.
***

`max_tokens` is a ceiling, and this stop reason means output was cut off. The
other options describe different failures, which show up as errors rather than a
truncated success.

***
```

Q2. Which TWO statements about a Messages API request are true?

```
[[ ]] `max_tokens` is the number of tokens Claude will aim to produce
[[X]] `system` is passed as a top-level parameter
[[ ]] `messages` may include a `role: "system"` entry
[[X]] `response.content` is a list of content blocks
[[ ]] Messages must all have the role `user`
[[?]] Is the system prompt a role or a parameter?
***

* `max_tokens` is a limit, not a target.
* The system prompt is a parameter, not a role.
* Roles alternate between `user` and `assistant`.

***
```

3. SDK Setup and Multi-turn State

Authentication and configuration

- The SDK reads `ANTHROPIC_API_KEY` from the environment by default.
- Keep the key in a `.env` file (loaded with `python-dotenv`) and **never commit it**.
- Official SDKs exist for Python, TypeScript, JavaScript, Go and Ruby. Plain HTTP also works.
- If you must pass the key explicitly: `Anthropic(api_key=...)`. Prefer the environment variable.
- A leaked key should be **revoked** in the console and replaced.

```python
from dotenv import load_dotenv
load_dotenv()                # populate os.environ from .env
```

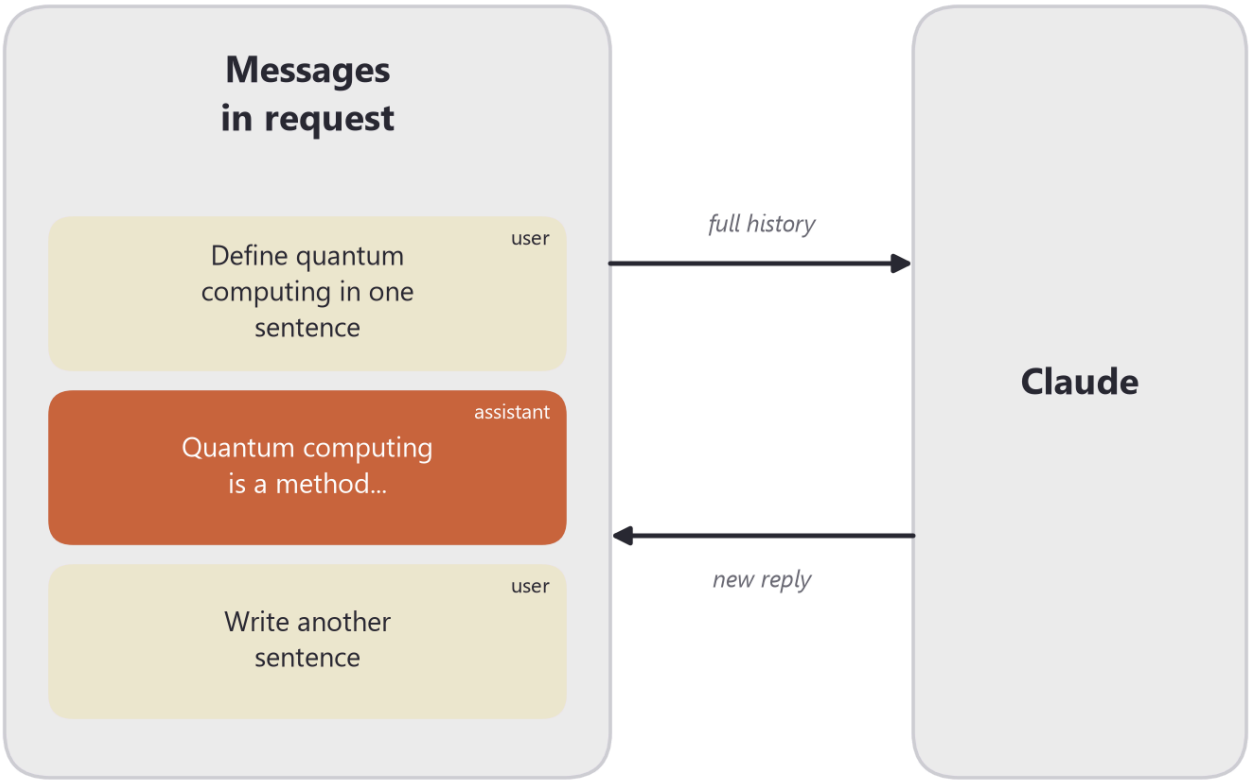
```
from anthropic import Anthropic
client = Anthropic()          # picks up ANTHROPIC_API_KEY
```

The API is stateless

Claude **does not store your conversation**. Every request is independent. Ask "What is quantum computing?", then ask "Write another sentence" in a separate request, and Claude has no idea what you mean.

To get a multi-turn conversation you must:

1. Keep a list of all messages in your own code.
2. Send the **complete history** with every request.



Helper functions

```
def add_user_message(messages, text):
    messages.append({"role": "user", "content": text})

def add_assistant_message(messages, text):
    messages.append({"role": "assistant", "content": text})

def chat(messages, system=None):
    params = {"model": model, "max_tokens": 1000, "messages": messages}
    if system:
        params["system"] = system
    message = client.messages.create(**params)
    return text_of(message)
```

```
messages = []

add_user_message(messages, "Define quantum computing in one sentence")
answer = chat(messages)
add_assistant_message(messages, answer)        # do not forget this step

add_user_message(messages, "Write another sentence")
final_answer = chat(messages)                  # Claude now has the context
```

Consequences of statelessness

Consequence	Why it matters
History grows each turn	Input tokens (and cost and latency) grow with it
Your app owns the state	If you drop a message, Claude "forgets" it
Long chats can hit the context limit	Plan to trim or summarise old turns
You can edit history	Useful for testing, but a mistake here changes behaviour

Worked example: state and authentication

The outputs are **illustrative** sample values.

Example 1: what the history looks like

```
import json

messages = []
add_user_message(messages, "Define quantum computing in one sentence")
answer = chat(messages)
add_assistant_message(messages, answer)
add_user_message(messages, "Write another sentence")

print(json.dumps(messages, indent=2))
```

```
[
  { "role": "user", "content": "Define quantum computing in one sentence" },
  { "role": "assistant", "content": "Quantum computing uses qubits and
superposition to solve certain problems much faster than classical computers." },
  { "role": "user", "content": "Write another sentence" }
]
```

Sending all three messages:

```
print(chat(messages))
```

```
Unlike classical bits, qubits can exist in several states at once, which lets
quantum
algorithms explore many possibilities in parallel.
```

Example 2: the same follow-up without history

```
print(chat([{"role": "user", "content": "Write another sentence"}]))
```

```
The old lighthouse stood quietly at the edge of the cliff as the waves rolled in
below.
```

Claude has no idea what "another" refers to, so it writes about something unrelated. This is the symptom from Q3.

Example 3: input tokens grow every turn

```
for turn in ["What is a qubit?", "And superposition?", "Give an analogy."]:
    add_user_message(messages, turn)
    message = client.messages.create(model=model, max_tokens=1000,
messages=messages)
    add_assistant_message(messages, text_of(message))
    print(len(messages), "messages ->", message.usage.input_tokens, "input
tokens")
```

```
2 messages -> 18 input tokens
4 messages -> 96 input tokens
6 messages -> 187 input tokens
```

Each request resends everything, so cost and latency rise with the length of the conversation.

Example 4: authentication errors

Key not set (`ANTHROPIC_API_KEY` missing or `.env` not loaded):

```
TypeError: "Could not resolve authentication method. Expected either api_key or
auth_token to be set. Or for one of the `X-Api-Key` or `Authorization` headers
```

```
to be explicitly omitted"
```

Key set but wrong or revoked (HTTP 401):

```
anthropic.AuthenticationError: Error code: 401 - {'type': 'error', 'error':
{'type':
'authentication_error', 'message': 'invalid x-api-key'}, 'request_id':
'req_01...'}
```

Fixes:

- For the first error, call `load_dotenv()` **before** creating `Anthropic()`, and check the `.env` file location.
- For the second, create a new key in the console. A key is shown only once, so a lost key must be replaced.

Quiz: SDK and state

Q3. A chatbot answers the first question correctly, but the follow-up "write another sentence" is about a random topic. What is the most likely cause?

```
[( )] The model is too small
[(X)] The prior messages were not included in the follow-up request
[( )] `max_tokens` was too high
[( )] Streaming was enabled
***

The API is stateless, so the history must be resent every turn.

***
```

Q4. Which TWO practices are correct for API key handling?

```
[[X]] Store the key in an environment variable or `.env` file kept out of version
control
[[ ]] Call the Anthropic API directly from browser JavaScript so there is no
server cost
[[X]] Route client requests through your own server that holds the key
[[ ]] Hard-code the key in the repository so teammates can run the code
[[ ]] Put the key in the `system` prompt so Claude can use it
[[?]] Which options keep the secret out of places other people can read?
***

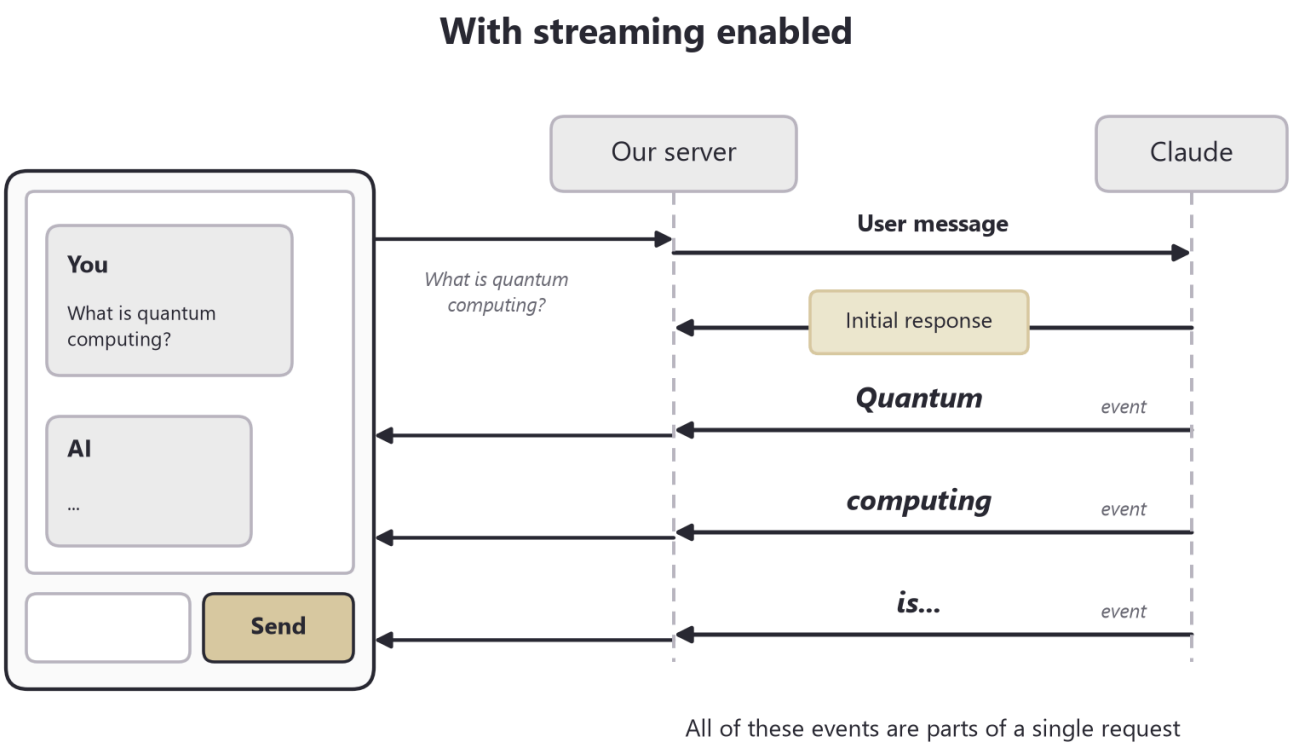
Browser code and committed repositories expose the key, and the system prompt is
not a place for secrets.

***
```

4. Streaming

Why stream

A full response can take 10–30 seconds. Without streaming, the user stares at a spinner. With streaming, text appears as it is generated. It is **one request**, and the server forwards chunks to the client as they arrive.



For comparison: a non-streaming request and response

Without `stream=True`, the call **blocks** until the whole reply is generated. Then you get one complete JSON object. The values below are **illustrative**.

```
import time

start = time.time()
message = client.messages.create(
    model=model,
    max_tokens=1000,
    messages=[{"role": "user", "content": "Write a 1 sentence description of a
fake database"}],
)
print(f"Returned after {time.time() - start:.1f}s")
print(text_of(message))
print(message.stop_reason)
```

```
Returned after 2.4s
NimbusDB is a distributed, schema-free database that stores every record as a
```

```
self-healing cloud of linked facts.
end_turn
```

The HTTP response body is a single JSON document:

```
POST /v1/messages HTTP/1.1
Host: api.anthropic.com
x-api-key: sk-ant-...
anthropic-version: 2023-06-01
content-type: application/json

{
  "model": "claude-sonnet-4-0",
  "max_tokens": 1000,
  "messages": [
    { "role": "user", "content": "Write a 1 sentence description of a fake
database" }
  ]
}
```

```
HTTP/1.1 200 OK
content-type: application/json

{
  "id": "msg_01NonStreamExample000",
  "type": "message",
  "role": "assistant",
  "model": "claude-sonnet-4-20250514",
  "content": [
    {
      "type": "text",
      "text": "NimbusDB is a distributed, schema-free database that stores every
record as a self-healing cloud of linked facts."
    }
  ],
  "stop_reason": "end_turn",
  "stop_sequence": null,
  "usage": { "input_tokens": 17, "output_tokens": 28 }
}
```

	Non-streaming	**Streaming**
Call	`messages.create(...)`	`messages.stream(...)` or `stream=True`
Response	One JSON `Message`	A series of events
User sees text	After the **whole** reply is done	As each chunk arrives

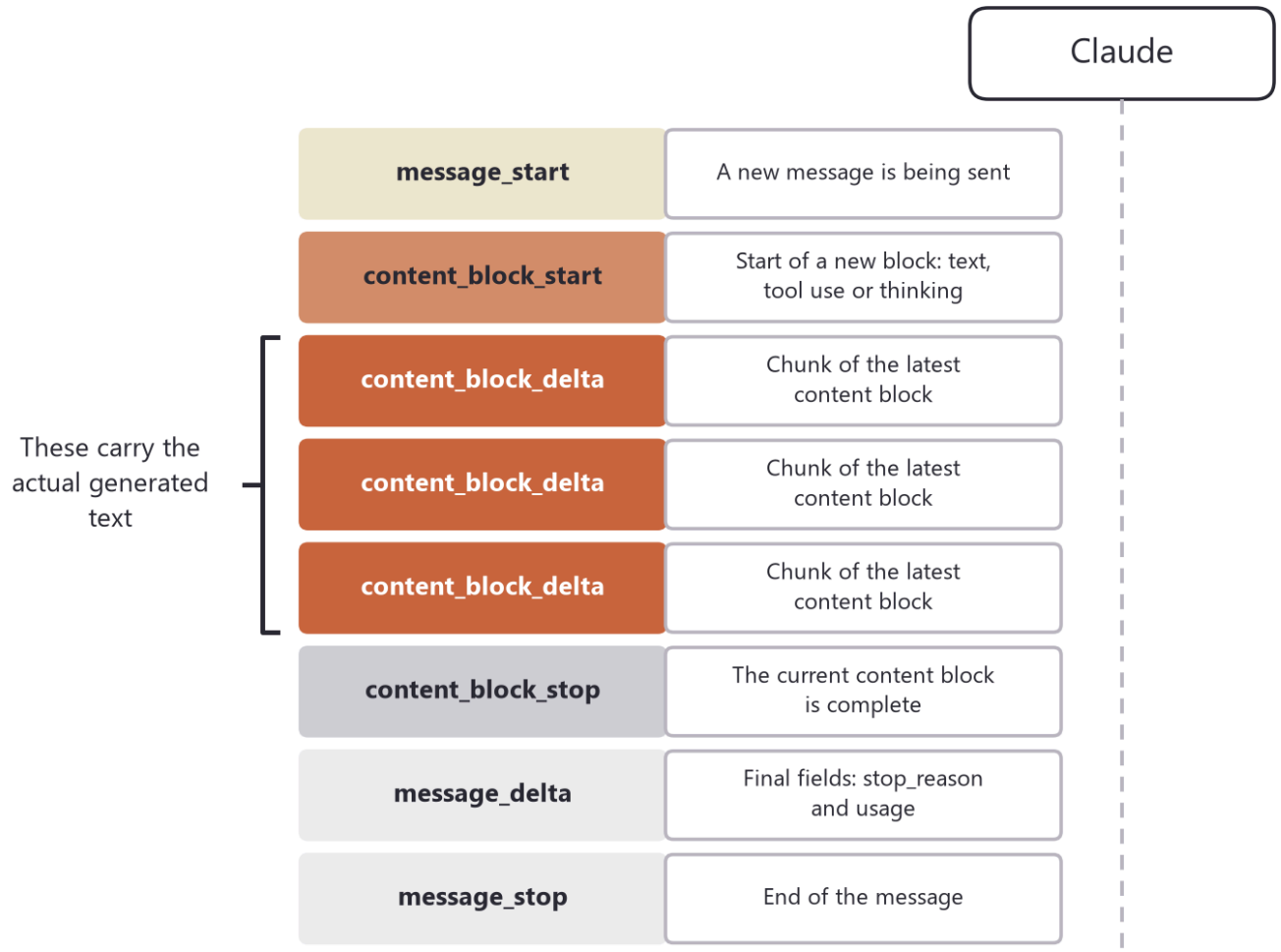
	Non-streaming	Streaming
`stop_reason`	In the response body	In the `message_delta` event
Best for	Short replies, batch jobs, back-end processing	Chat UIs, long replies

For a long reply, the user waits for the full generation time before seeing anything. With streaming, the first words appear within a moment, and the total time is about the same.

Event types

Event	Meaning
`message_start`	A new message begins
`content_block_start`	A new content block begins (text, tool use, ...)
`content_block_delta`	A chunk of the block's content. **This carries the text you display.**
`content_block_stop`	The current block is complete
`message_delta`	Top-level message changes, including the final `stop_reason` and usage
`message_stop`	The message is complete

The API may also send `ping` and `error` events. Handle them without crashing.



Order for a simple text reply

```
message_start
  content_block_start
    content_block_delta   (x many)
  content_block_stop
message_delta             <- stop_reason arrives here
message_stop
```

Delta types inside `content_block_delta`: `text_delta` for text, `input_json_delta` for partial tool-call JSON, `thinking_delta` for extended thinking. A model that thinks first sends `thinking_delta` events **before** the first `text_delta`, so check `delta.type` before reading `.text`.

Handling partial output

Raw events (`stream=True`): you parse each event yourself.

```
stream = client.messages.create(
    model=MODEL, max_tokens=2000, stream=True,
    messages=[{"role": "user", "content": "Say hello in two words."}],
)

for event in stream:
    if event.type == "content_block_delta":
        d = event.delta
        field = {"text_delta": "text", "thinking_delta": "thinking",
"input_json_delta": "partial_json"}.get(d.type)
        print(f"{event.type:22} {d.type}: {getattr(d, field) if field else
'...'!r}")
    elif event.type == "message_delta":
        print(f"{event.type:22} stop_reason={event.delta.stop_reason},
output_tokens={event.usage.output_tokens}")
    else:
        print(event.type)
```

Response output:

```
message_start
content_block_start
content_block_delta    text_delta: 'Hello there'
content_block_delta    text_delta: '!'
content_block_stop
message_delta          stop_reason=end_turn, output_tokens=7
message_stop
```

Helper (`messages.stream`): the SDK extracts the text for you.

```python
with client.messages.stream(
    model=MODEL, max_tokens=2000,
    messages=[{"role": "user", "content": "Explain streaming in two sentences."}],
) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)           # flush=True or the terminal
buffers it

    final = stream.get_final_message()             # inside the with-block

print()
print("stop_reason:", final.stop_reason, "| output_tokens:",
final.usage.output_tokens)
print("assembled  :", text_of(final)[:60] + "...")
```

Response output:

```
Streaming is the continuous delivery of data, such as audio, video, or text, in a
steady flow so it can be played or processed as it arrives, rather than waiting
for the entire file to download first. This approach allows users to begin
consuming content almost immediately while the rest is still being transmitted in
the background.
stop_reason: end_turn | output_tokens: 95
assembled  : Streaming is the continuous delivery of data, such as audio,...
```

`final_message` is the assembled `Message`. Here its `stop_reason` is `end_turn`, and `text_of(final_message)` returns the whole sentence above in one string.

Use the helper unless you need individual events. Use `get_final_message()` when you need the complete message afterwards, for example to save it to a database or append it to the history.

Partial output pitfalls

- **Do not treat partial text as final.** A stream can end early through an error or a `max_tokens` stop. Read `stop_reason` after the stream, not before.
- **Tool-call JSON arrives in fragments** (`input_json_delta`). Accumulate it and parse only after the block stops.
- **Flush output** (`flush=True`) or the terminal buffers it and streaming looks broken.
- **Leave room for thinking.** On a model that thinks first, the first text arrives later, and a small `max_tokens` can produce no text at all. Check `stop_reason`.

Worked example: streaming output

The outputs are **illustrative**. IDs, text and token counts are sample values.

Example 1: the raw event stream

```python
with client.messages.stream(
    model=model, max_tokens=1000,
    messages=[{"role": "user", "content": "Say hello in two words."}],
) as stream:
    for event in stream:
        print(event.type)
```

On the wire, the server sends these server-sent events:

```
event: message_start
data: {"type":"message_start","message":
{"id":"msg_01Abc...","type":"message","role":"assistant","content":
[],"model":"claude-sonnet-4-
20250514","stop_reason":null,"stop_sequence":null,"usage":
{"input_tokens":12,"output_tokens":1}}}

event: content_block_start
data: {"type":"content_block_start","index":0,"content_block":
{"type":"text","text":""}}

event: ping
data: {"type":"ping"}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":
{"type":"text_delta","text":"Hello"}}

event: content_block_delta
data: {"type":"content_block_delta","index":0,"delta":
{"type":"text_delta","text":" there!"}}

event: content_block_stop
data: {"type":"content_block_stop","index":0}

event: message_delta
data: {"type":"message_delta","delta":
{"stop_reason":"end_turn","stop_sequence":null},"usage":{"output_tokens":6}}

event: message_stop
data: {"type":"message_stop"}
```

What to notice:

- `message_start` has an **empty** `content` list and `stop_reason: null`. Nothing is known yet.
- Each `text_delta` holds only a fragment. Join them to get "Hello there!".
- The real `stop_reason` and final `output_tokens` arrive in `message_delta`, near the end.
- `index` says which content block a delta belongs to.

Example 2: `text_stream` and the final message

```python
with client.messages.stream(model=model, max_tokens=1000, messages=messages) as
stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
    final = stream.get_final_message()

print()
print(final.stop_reason, final.usage.output_tokens)
```

```
Hello there!
end_turn 6
```

The chunks printed one at a time ("Hello", then " there!"). `get_final_message()` returns the assembled `Message`, with `text_of(final) == "Hello there!"`.

Example 3: streaming a tool call

When Claude calls a tool, the arguments arrive as JSON **fragments** in `input_json_delta` events.

```
event: content_block_start
data: {"type":"content_block_start","index":1,"content_block":
{"type":"tool_use","id":"toolu_01A...","name":"get_weather","input":{}}}

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":
{"type":"input_json_delta","partial_json":"{\"ci"}}

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":
{"type":"input_json_delta","partial_json":"ty\": \"Chen"}}

event: content_block_delta
data: {"type":"content_block_delta","index":1,"delta":
{"type":"input_json_delta","partial_json":"nai\"}"}}

event: content_block_stop
data: {"type":"content_block_stop","index":1}
```

Joined, the fragments make `{"city": "Chennai"}`. Each fragment on its own (`{"ci`) is **not** valid JSON, so parse only after `content_block_stop`.

Quiz: Streaming

Q5. Which event carries the generated text a user should see?

```
[( )] `message_start`
[( )] `content_block_start`
[(X)] `content_block_delta`
[( )] `message_stop`
***

Deltas carry the content. The start and stop events only mark boundaries.

***
```

Q6. Which TWO statements about streaming are true?

```
[[ ]] Streaming sends one request per chunk of text
[[X]] `message_delta` is where the final `stop_reason` is reported
[[X]] `get_final_message()` returns the assembled message after the stream ends
[[ ]] Streaming removes the need for `max_tokens`
[[ ]] Partial tool-call JSON can be parsed safely as soon as each delta arrives
[[?]] Is it one request or many?
[[?]] Is partial JSON valid JSON?
***

* It is a single request.
* `max_tokens` is still required.
* The JSON is incomplete until the block stops.

***
```

5. Hands-On Lab

Goal: build a streaming, multi-turn chat client in the terminal. Work in pairs (one drives, one reviews), then swap.

Starter file: `chat_client.py`

```python
from dotenv import load_dotenv
load_dotenv()

from anthropic import Anthropic

client = Anthropic()
model = "claude-sonnet-4-0"

def text_of(message):
    """Join the text blocks. content[0] is not always text: it can be a thinking
or tool_use block."""
    return "".join(b.text for b in message.content if b.type == "text")

SYSTEM = "You are a concise technical assistant."   # TODO 4: experiment with this
```

```
messages = []

def add_user_message(messages, text):
    messages.append({"role": "user", "content": text})

def add_assistant_message(messages, text):
    messages.append({"role": "assistant", "content": text})

def stream_reply(messages):
    # TODO 1: use client.messages.stream(...) with system=SYSTEM
    # TODO 2: print each chunk from stream.text_stream with end="" and flush=True
    # TODO 3: return the assembled text, and print the stop_reason afterwards
    pass

while True:
    user_input = input("\nYou: ")
    if user_input.strip().lower() in {"exit", "quit"}:
        break
    add_user_message(messages, user_input)
    print("Claude: ", end="")
    reply = stream_reply(messages)
    add_assistant_message(messages, reply)      # keep the history in sync
```

Tasks

- [ ] Implement `stream_reply` with `messages.stream` and `text_stream`
- [ ] Return the final text and print `stop_reason` after each reply
- [ ] Confirm a follow-up question uses earlier context
- [ ] Change the system prompt (tutor, pirate, terse) and compare
- [ ] Set `max_tokens=30`, ask for a long answer, and observe `stop_reason`
- [ ] Comment out `add_assistant_message` and observe what breaks

Reference solution

```
def stream_reply(messages):
    with client.messages.stream(
        model=model,
        max_tokens=1000,
        system=SYSTEM,
        messages=messages,
    ) as stream:
        for text in stream.text_stream:
            print(text, end="", flush=True)
        final = stream.get_final_message()

    print(f"\n[stop_reason: {final.stop_reason}]")
    return text_of(final)
```

Expected output

An illustrative terminal session when the reference solution works. Your wording will differ.

```
You: What is a token?
Claude: A token is a chunk of text, often a word or part of a word, that the model
reads and writes.
[stop_reason: end_turn]

You: Give me an example
Claude: The word "unbelievable" might be split into "un", "believ" and "able",
which is three tokens.
[stop_reason: end_turn]

You: exit
```

What to check:

- The second answer builds on the first, so the history is working.
- `[stop_reason: end_turn]` prints after each reply.
- With `max_tokens=30` you would instead see a reply cut off, followed by `[stop_reason: max_tokens]`.
- With `add_assistant_message` commented out, the second answer loses the context and drifts off topic.

Stretch goals

- Use raw `stream=True` events and print only `text_delta`s.
- Print token usage (`final.usage.input_tokens`, `final.usage.output_tokens`) per turn and watch the input tokens grow.
- Handle `stop_reason == "max_tokens"` by telling the user the reply was cut off.

Exam-style questions

Work through Q1–Q6 again **without** your notes, then try these.

Q7. A streaming client prints partial text correctly but crashes when saving the final reply. Which TWO fixes are appropriate?

```
[[X]] Call `get_final_message()` after the stream finishes
[[X]] Concatenate the received `text_delta` chunks into one string
[[ ]] Re-send the request without streaming to get the text
[[ ]] Remove `max_tokens`
[[ ]] Add a `role: "system"` message to the history
[[?]] You already have the text. How do you gather it?
***

Either assemble the text yourself or take it from the final message. A second
request wastes tokens, `max_tokens` is required, and `system` is not a role.
```

```
***
```

Q8. After a streaming call you need to know whether the reply was truncated. Where do you look?

```
[( )] The first `message_start` event
[(X)] The `stop_reason` reported by `message_delta` (or on the final message)
[( )] The length of the last text chunk
[( )] The API key permissions
***

The stop reason is not known until generation ends.

***
```

6. Wrap-Up

Cheat sheet

Request: `model` + `messages` + `max_tokens` are required. `system` is optional, top-level, and never `None`.

Roles: `user` and `assistant`, alternating. System is a parameter, not a role.

Response: `content` is a **list of blocks**. Join the `text` blocks (`text_of`); `content[0]` can be a `thinking` block. Also `stop_reason` and `usage`.

`stop_reason`: `end_turn` · `max_tokens` (truncated) · `stop_sequence` · `tool_use` · `pause_turn` · `refusal`.

Streaming events: `message_start` → `content_block_start` → `content_block_delta` ×N → `content_block_stop` → `message_delta` (stop_reason) → `message_stop`.

Streaming code: `client.messages.stream()` + `.text_stream` + `.get_final_message()`.

State: the API is stateless. Append every user **and** assistant message and resend the whole history.

Security: keep the key in `ANTHROPIC_API_KEY` or `.env`, never in client code or git. Requests go through your server.

Preview of Session 2

Temperature, structured data output, and prompt evaluation.